

Concise Report on Three-Structure LSTM-Based Dynamic System Identification of Actuator Response

Hussein Zolfaghari

July 23, 2026

Abstract

This report summarizes a nonlinear dynamic-system identification study in which coil current is used to predict actuator displacement and Lorentz force. Three structures are implemented and compared: series, series-parallel, and parallel/free-running prediction. Four DC-offset experiments are used for training and validation, while the 147 mA experiment is kept completely unseen for final testing.

1 Purpose and Main Idea

The known input vector is

$$u(k) = [I(k) \quad \Delta I(k) \quad I_{\text{DC}}]^T,$$

and the output vector is

$$y(k) = [x(k) \quad F(k)]^T,$$

where I is coil current, x is displacement, and F is Lorentz force.

A history of 120 samples is used to predict the next output. The three implemented structures are

Table 1: Implemented prediction structures

Structure	Mathematical input-output relation
Series	$[I, \Delta I, I_{\text{DC}}]_{\text{past}} \rightarrow [\hat{x}(k+1), \hat{F}(k+1)]$
Series-parallel	$[I, \Delta I, I_{\text{DC}}, x, F]_{\text{past}} \rightarrow [\hat{x}(k+1), \hat{F}(k+1)]$
Parallel	$[I, \Delta I, I_{\text{DC}}, \hat{x}, \hat{F}]_{\text{past}} \rightarrow [\hat{x}(k+1), \hat{F}(k+1)]$

2 Connection to the Reference Papers

Connection to Wang (2017)

The implementation uses Wang’s distinction between series-parallel identification, where measured output history is supplied, and parallel identification, where predicted output history is fed back. The present code does not reproduce Wang’s complete convex multi-LSTM ensemble; it is therefore a Wang-inspired implementation.

Connection to Ogunmolu et al. (2016)

The model follows the general idea of using deep dynamic neural networks for nonlinear input–output identification. A static nonlinear transformation of the known inputs is followed by recurrent LSTM dynamics, giving a Hammerstein-style neural architecture. The code is a PyTorch implementation, not a direct conversion of the original FARNN source code.

3 Dataset and Data Usage

The development datasets are

67, 87, 107, 127 mA,

and the independent test dataset is

147 mA.

Cyclic blocked split across the chirp

Because chirp frequency changes continuously, a single contiguous split can make one subset contain mostly low-frequency behavior and another subset contain mostly high-frequency behavior. The updated code therefore uses a cyclic blocked split inside each development record:

train → guard gap → validation → guard gap → development test → guard gap,

and then repeats this pattern across the full chirp record. This makes the training, validation, and internal development-test blocks cover different frequency regions while reducing leakage from overlapping LSTM windows. The complete 147 mA record is still reserved only for the final unseen test.

The current implementation uses 1000 training samples, 500 validation samples, 500 internal development-test samples, and a 120-sample guard gap in each cycle. Normalization statistics are calculated from training blocks only.

Cyclic split schematic

Insert the figure generated by the Python code.

Expected filename: 00_cyclic_split_schematic.png

Copy it from 01_Report_Figures/<timestamp>/ into figures/simulation/.

Figure 1: Schematic of the repeated train–validation–development-test block pattern used across each chirp record.

Training, validation, development-test, and final-test data usage

Insert the figure generated by the Python code.

Expected filename: 02_data_usage_summary.png

Copy it from 01_Report_Figures/<timestamp>/ into figures/simulation/.

Figure 2: Cyclic blocked split across the four development experiments. The complete 147 mA record is reserved for final testing.

The code also produces colored current plots for every operating condition:

- blue: training blocks;
- orange: validation blocks;
- purple: internal development-test blocks;
- gray: guard-gap samples not used for optimization;
- green: independent 147 mA final test record.

4 Model Architecture and Training

Architecture summary

The known inputs first pass through a static nonlinear block

$$3 \rightarrow 24 \rightarrow 24$$

with Tanh activation. The dynamic block is an LSTM with 64 hidden units. The final prediction head is

$$64 \rightarrow 32 \rightarrow 2.$$

The two outputs are displacement and Lorentz force.

Training summary

The series and series-parallel models are trained for 15 epochs using Adam, a learning rate of 10^{-3} , weight decay, gradient clipping, and validation checkpoint selection. The series-parallel model receives measured output history, with small training noise added for robustness. The parallel model starts from the series-parallel weights and is fine-tuned for 8 epochs using 50-step recursive rollouts. The measured-feedback ratio decreases from 0.9 to 0.1.

5 Simulation Results

The figures below are generated directly by the presentation-ready Python simulation. All prediction results use the same completely unseen 147 mA chirp. The first 120 samples are used only to initialize feedback-based evaluation and are excluded from the reported accuracy metrics.

5.1 Training and Validation Behavior

Training and validation cost

Insert the figure generated by the Python code.

Expected filename: 01_all_structures_cost.png

Copy it from 01.Report.Figures/<timestamp>/ into figures/simulation/.

Figure 3: Normalized training and validation MSE for series, series-parallel, and parallel models. The parallel objective is more difficult because it includes recursive multi-step prediction.

5.2 Overall Comparison on the Unseen Test

The new comparison figures contain both the full test record and a zoomed high-frequency region. The zoom prevents important local differences from being hidden by the full-record scale.

Displacement comparison for all structures

Insert the figure generated by the Python code.

Expected filename: 03_displacement_all_structures.png

Copy it from 01.Report.Figures/<timestamp>/ into figures/simulation/.

Figure 4: Measured displacement together with series, series-parallel, and parallel predictions over the full unseen record and a high-frequency zoom.

Lorentz-force comparison for all structures

Insert the figure generated by the Python code.

Expected filename: 03_force_all_structures.png

Copy it from 01.Report.Figures/<timestamp>/ into figures/simulation/.

Figure 5: Measured Lorentz force together with series, series-parallel, and parallel predictions over the full unseen record and a high-frequency zoom.

5.3 Structure-by-Structure Tracking

The following figures are the clearest way to compare the three strategies, because each structure is plotted against the measured response without being compressed by a strongly diverging curve from another model.

Series measured versus predicted response

Insert the figure generated by the Python code.

Expected filename: 06_series_measured_vs_predicted.png

Copy it from 01_Report_Figures/<timestamp>/ into figures/simulation/.

Figure 6: Series/input-only prediction on the unseen 147 mA chirp. This model is the baseline because it uses known input history but no output history.

Series-parallel measured versus predicted response

Insert the figure generated by the Python code.

Expected filename: 06_series_parallel_measured_vs_predicted.png

Copy it from 01_Report_Figures/<timestamp>/ into figures/simulation/.

Figure 7: Series-parallel one-step prediction using measured displacement and force history. This figure demonstrates the expected accuracy advantage of measured output feedback.

Parallel measured versus predicted response

Insert the figure generated by the Python code.

Expected filename: 06_parallel_measured_vs_predicted.png

Copy it from 01_Report_Figures/<timestamp>/ into figures/simulation/.

Figure 8: Parallel/free-running prediction after initialization. Predicted displacement and force are fed back recursively, so phase, amplitude, and bias errors can accumulate.

Main comparison

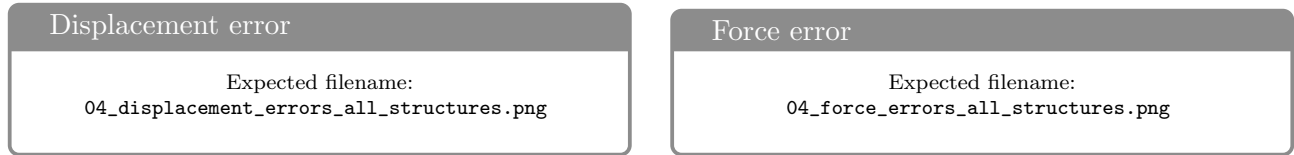
The series model provides the input-only baseline. The series-parallel model is expected to give the most accurate one-step prediction because measured past outputs are supplied at every step. The parallel model is the strongest test of autonomous simulation because it must continue using its own predicted output history. Therefore, the principal advantage of the series-parallel strategy is more stable learning and more accurate short-horizon tracking, while the parallel result reveals long-horizon recursive stability.

5.4 Prediction Errors

The error is defined as

$$e(k) = y(k) - \hat{y}(k).$$

Direct error plots are more meaningful than a crowded response plot when one structure has much larger drift than the others.



(a) Displacement error histories (b) Lorentz-force error histories

Figure 9: Prediction errors for all three structures on the same unseen test record.

5.5 Regression and Accuracy Metrics

The regression figures plot predicted values against measured values. Ideal prediction follows

$$\hat{y} = y.$$

A slope close to one, an intercept close to zero, and regression R^2 close to one indicate strong agreement.

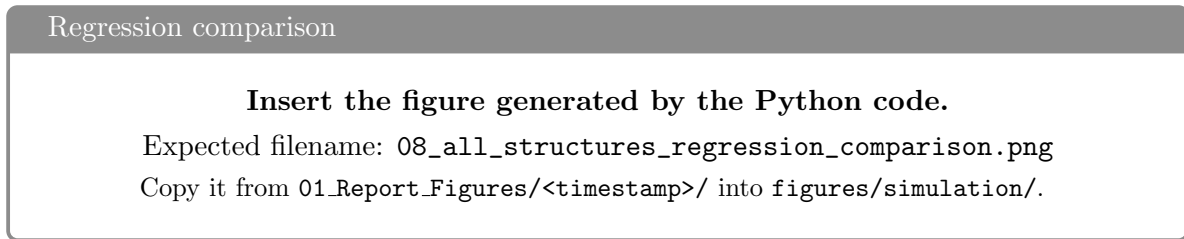


Figure 10: Measured-versus-predicted regression plots for displacement and Lorentz force for series, series-parallel, and parallel models.

The simulation also generates individual regression figures:

- 07_series_regression.png;
- 07_series_parallel_regression.png;
- 07_parallel_regression.png.

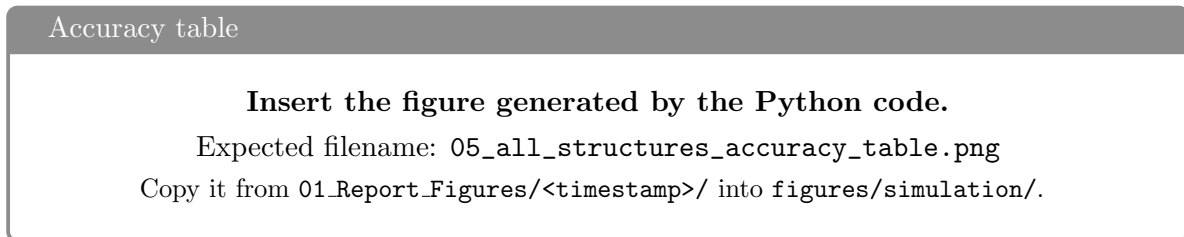


Figure 11: RMSE, MAE, R^2 , and system-identification fit percentage for displacement and Lorentz force for all three structures.

How to interpret the final result

The advantage of series-parallel prediction should be demonstrated using the measured-versus-predicted tracking figure, the error histories, the regression plot, and the numerical table together. A visually good curve alone is not sufficient. Similarly, weaker parallel performance should not be described as a failure of one-step identification; it indicates that recursive free-

running simulation is a substantially harder task.

6 Generated Files

Run the final simulation file first:

```
Three_Structures_LSTM_VSCode_Windows_Fixed_AutoPush_PresentationReady.py
```

After each successful run, the code creates

- 01_Report_Figures/<timestamp>;
- 02_Presentation_Figures/<timestamp>;
- 03_Complete_Results/<timestamp>.

The complete numerical outputs include

- all_structures_metrics.csv;
- 147mA_all_structures_predictions.csv;
- training_history.csv;
- data_split_blocks.csv;
- the three trained model files.

7 Summary

The final simulation compares series, series-parallel, and parallel identifiers using the same unseen 147 mA experiment. Cyclic non-overlapping blocks expose training, validation, and internal development testing to different chirp-frequency regions while guard gaps reduce window leakage. The updated result set includes full-record and high-frequency comparisons, separate measured-versus-predicted figures for each structure, direct error plots, regression figures, and numerical accuracy metrics. These outputs make the expected advantage of series-parallel one-step identification clear while keeping parallel/free-running evaluation as the stricter autonomous test.

References

- [1] Y. Wang, “A New Concept using LSTM Neural Networks for Dynamic System Identification,” American Control Conference, 2017.
- [2] O. Ogunmolu, X. Gu, S. Jiang, and N. Gans, “Nonlinear Systems Identification Using Deep Dynamic Neural Networks,” arXiv:1610.01439, 2016.
- [3] C. Bergmeir, R. J. Hyndman, and B. Koo, “A Note on the Validity of Cross-Validation for Evaluating Autoregressive Time Series Prediction,” Computational Statistics & Data Analysis, 2018.
- [4] V. Cerqueira, L. Torgo, and I. Mozetič, “Evaluating Time Series Forecasting Models: An Empirical Study on Performance Estimation Methods,” Machine Learning, 2020.